

RETO HORA

POO. La clase Hora:

- a. Diseñar una clase que se llame Hora que va a representar un instante de tiempo por una hora y minutos.
 - Dispone del constructor por defecto: la hora y los minutos son 0
 - Dispone del constructor con horas y minutos: Hora (hora, minutos)
 - o En este constructor se llamará al método de setHora(hora) y setMinuto(minuto) para controlar los siguientes casos de que la hora y los minutos se pongan correctamente, hora del 0 al 23 y minutos de 0 a 59
 - Dispone de todos los métodos para acceder a los atributos
 - Dispone de un método inc, que incrementa un minuto. Hay que controlar el paso de minutos a horas
 - Haz un overriding del método toString, para que ponga hora:minutos
- b. Realiza un programa principal donde se vea el funcionamiento de la clase, para ello introduce la hora de las 11:30 y repite el incremento 40 veces y muestra el resultado. Luego cambia solo la hora a 20
- c. Crear una clase Hora12, que funciona de una forma similar a Hora, con la diferencia de que las horas solo pueden tomar los valores de 1 al 12, y se distingue la mañana y la tarde con el String de am o pm. Haz un override de setHora(), inc() y el toString

Desarrollo de la Clase Hora y su Versión Hora12

1. Planteamiento del Problema

El enunciado nos pide que diseñemos una clase capaz de representar una hora con minutos. Esta clase debe permitir:

- Crear una hora, tanto por defecto como indicando valores concretos.
- Modificar la hora y el minuto de forma individual.
- Sumar un minuto mediante un método inc(), controlando correctamente los saltos de minuto (por ejemplo, de 59 a 00) y de hora (por ejemplo, de 23 a 00).
- Mostrar la hora de manera legible utilizando el método toString().

Además, se requiere una versión especial, denominada **Hora12**, que gestione el formato de 12 horas (de 1 a 12) junto con los indicadores "am" y "pm". Para ello, será necesario sobrescribir (override) algunos métodos para adaptar las reglas específicas de esta versión.

2. Definición de los Atributos

Para que el objeto reloj sea funcional, los atributos mínimos que necesita son:

- **hora**
- **minuto**

Estos atributos se declaran como **private**, garantizando así la encapsulación. Esto significa que nadie puede modificar directamente estos valores desde fuera de la clase, sino que se accede a ellos exclusivamente a través de métodos específicos (getters y setters).

3. Creación de Constructores

Es habitual proporcionar dos tipos de constructores:

- Un constructor por defecto, que inicializa la hora a 00:00.
- Un constructor con parámetros, que permite crear una hora concreta especificando los valores deseados.

4. Getters y Setters

Al ser los atributos *private*, es imprescindible definir métodos para poder leer y modificar sus valores:

- **getHora()** y **getMinuto()** para acceder a los valores.
- **setHora()** y **setMinuto()** para modificarlos.

En estos métodos es donde se debe incluir la **validación** de los datos para asegurar que los valores sean correctos:

- En la clase **Hora**, la hora debe estar entre 0 y 23.
- Los minutos deben estar siempre entre 0 y 59.

5. Programación del Comportamiento: inc()

El método inc() es el encargado de incrementar la hora en un minuto, siguiendo estas reglas:

- Suma un minuto al valor actual.
- Si los minutos alcanzan 60, se reinician a 0 y se incrementa la hora.
- Si la hora llega a 24, se reinicia a 0.

Este método es un claro ejemplo de **control de condiciones** dentro de la programación.

6. Método toString() para Representación

Para mostrar la hora de forma legible y estética al imprimir el objeto (por ejemplo, con `System.out.println(h);`), es necesario sobrescribir el método `toString()` para que devuelva la hora en formato "hh:mm". De esta forma, la salida será clara y fácil de entender, como "12:10", en lugar de la representación por defecto de Java.

Clase Hora12 derivada de Hora

Herencia: creación de Hora12 a partir de Hora

Se utiliza herencia, de modo que la clase **Hora12** extiende a la clase **Hora**. Así, en lugar de reescribir toda la funcionalidad, se aprovechan las características ya definidas en la clase base.

- `Hora12 extends Hora`

Este enfoque permite reutilizar elementos como:

- minutos
- validación de minutos
- estructura general de la clase

Sin embargo, dado que las reglas de funcionamiento cambian al trabajar con el formato de 12 horas, es necesario sobrescribir (override) ciertos métodos que controlan el comportamiento específico:

- `setHora()`
- `inc()`
- `toString()`

Encapsulación y acceso a los atributos

En la clase **Hora** los atributos son *private*. Esto impide acceder directamente a las variables de hora y minuto, por lo que **no se puede** hacer operaciones como *hora++* o *minuto++* en la subclase.

La solución consiste en utilizar los métodos de acceso definidos en la clase base:

- `getHora()`
- `setHora()`
- `getMinuto()`
- `setMinuto()`

De este modo, se **respeta la encapsulación** y se garantiza que el acceso y la modificación de los datos internos se realiza de forma controlada.

Conversión y salto de hora en formato 12h

Para gestionar correctamente el salto de hora en el formato de 12 horas, se utiliza la siguiente fórmula:

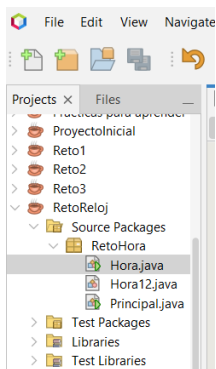
- Si $hora = 12 \rightarrow (12 \% 12) + 1 = 0 + 1 = 1$
- Si $hora = 11 \rightarrow (11 \% 12) + 1 = 11 + 1 = 12$
- Si $hora = 5 \rightarrow$

Clase Principal (probar el reto)

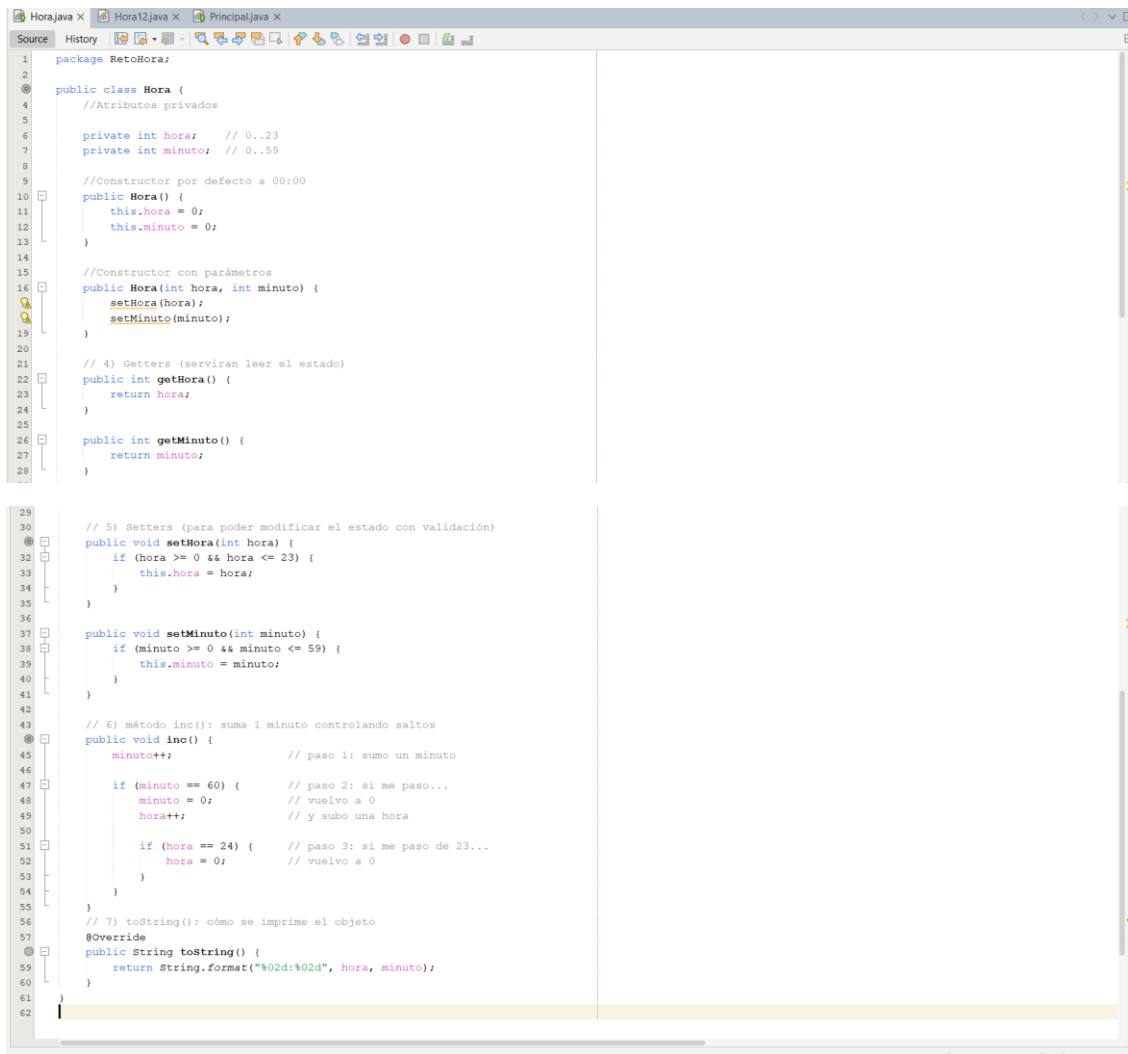
Qué hace

- crea `Hora(11, 30)`
- incrementa 40 minutos
- imprime
- cambia la hora a 20
- imprime
- ejemplo de `Hora12`

Creación clases



CLASE HORA



CÓDIGO JAVA:

```
package RetoHora;
public class Hora {
    //Atributos privados
    private int hora; // 0..23
    private int minuto; // 0..59

    //Constructor por defecto a 00:00
    public Hora() {
        this.hora = 0;
        this.minuto = 0;
    }
    //Constructor con parámetros
    public Hora(int hora, int minuto) {
        setHora(hora);
        setMinuto(minuto);
    }
    // 4) Getters (serviran leer el estado)
    public int getHora() {
        return hora;
    }
    public int getMinuto() {
        return minuto;
    }
    // 5) Setters (para poder modificar el estado con validación)
    public void setHora(int hora) {
        if (hora >= 0 && hora <= 23) {
            this.hora = hora;
        }
    }
    public void setMinuto(int minuto) {
        if (minuto >= 0 && minuto <= 59) {
            this.minuto = minuto;
        }
    }
    // 6) método inc(): suma 1 minuto controlando saltos
    public void inc() {
        minuto++; // paso 1: sumo un minuto

        if (minuto == 60) { // paso 2: si me paso...
            minuto = 0; // vuelvo a 0
            hora++; // y subo una hora

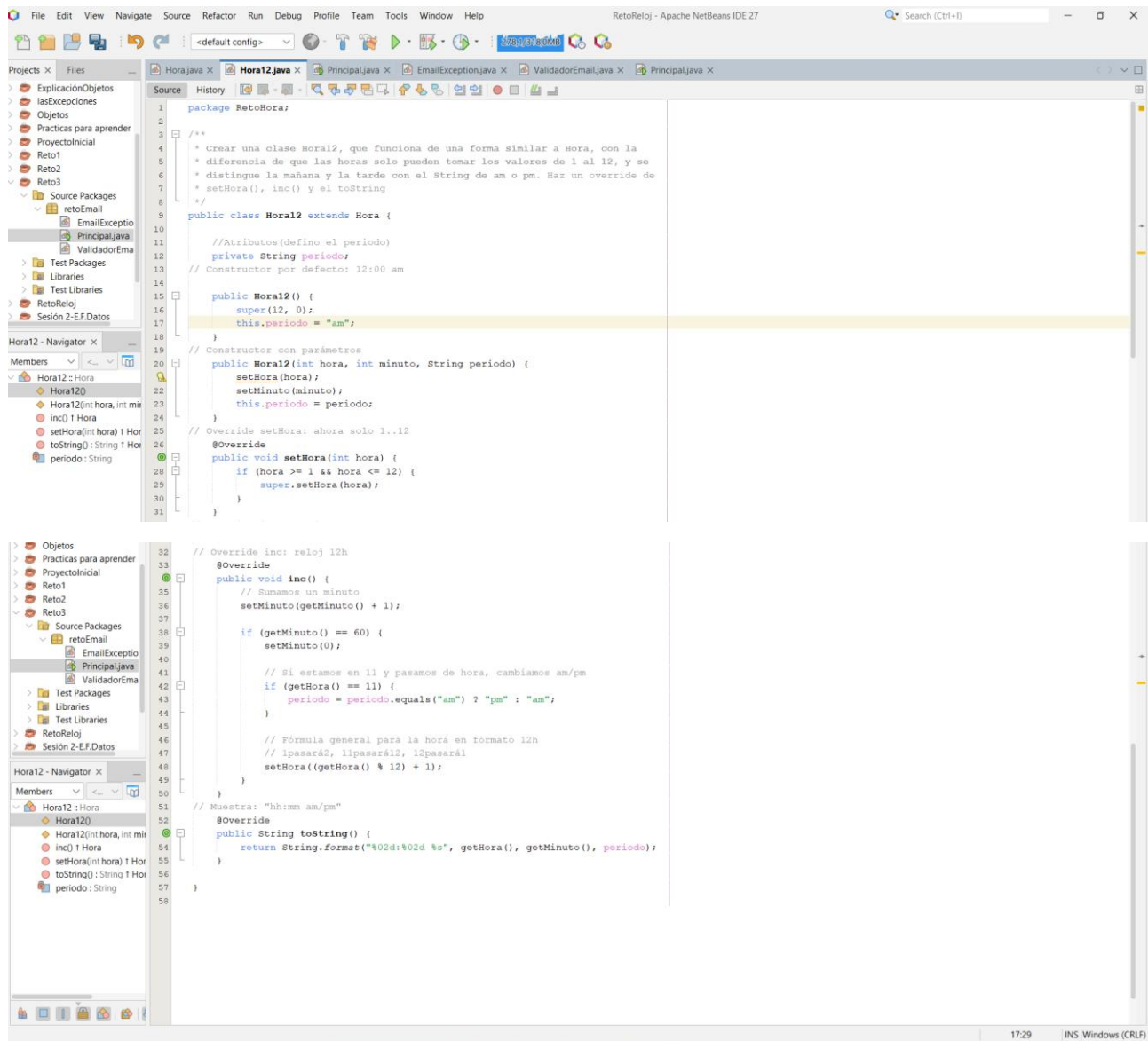
            if (hora == 24) { // paso 3: si me paso de 23...
```

```

        hora = 0;    // vuelvo a 0
    }
}
// 7) toString(): cómo se imprime el objeto
@Override
public String toString() {
    return String.format("%02d:%02d", hora, minuto);
}
}

```

CLASE HORA12



CÓDIGO JAVA:

```
package RetoHora;
/**
 * Crear una clase Hora12, que funciona de una forma similar a Hora, con la
 * diferencia de que las horas solo pueden tomar los valores de 1 al 12, y se
 * distingue la mañana y la tarde con el String de am o pm. Haz un override de
 * setHora(), inc() y el toString
 */
public class Hora12 extends Hora {

    //Atributos(defino el periodo)
    private String periodo;
    // Constructor por defecto: 12:00 am
    public Hora12() {
        super(12, 0);
        this.periodo = "am";
    }
    // Constructor con parámetros
    public Hora12(int hora, int minuto, String periodo) {
        setHora(hora);
        setMinuto(minuto);
        this.periodo = periodo;
    }
    // Override setHora: ahora solo 1..12
    @Override
    public void setHora(int hora) {
        if (hora >= 1 && hora <= 12) {
            super.setHora(hora);
        }
    }

    // Override inc: reloj 12h
    @Override
    public void inc() {
        // Sumamos un minuto
        setMinuto(getMinuto() + 1);

        if (getMinuto() == 60) {
            setMinuto(0);

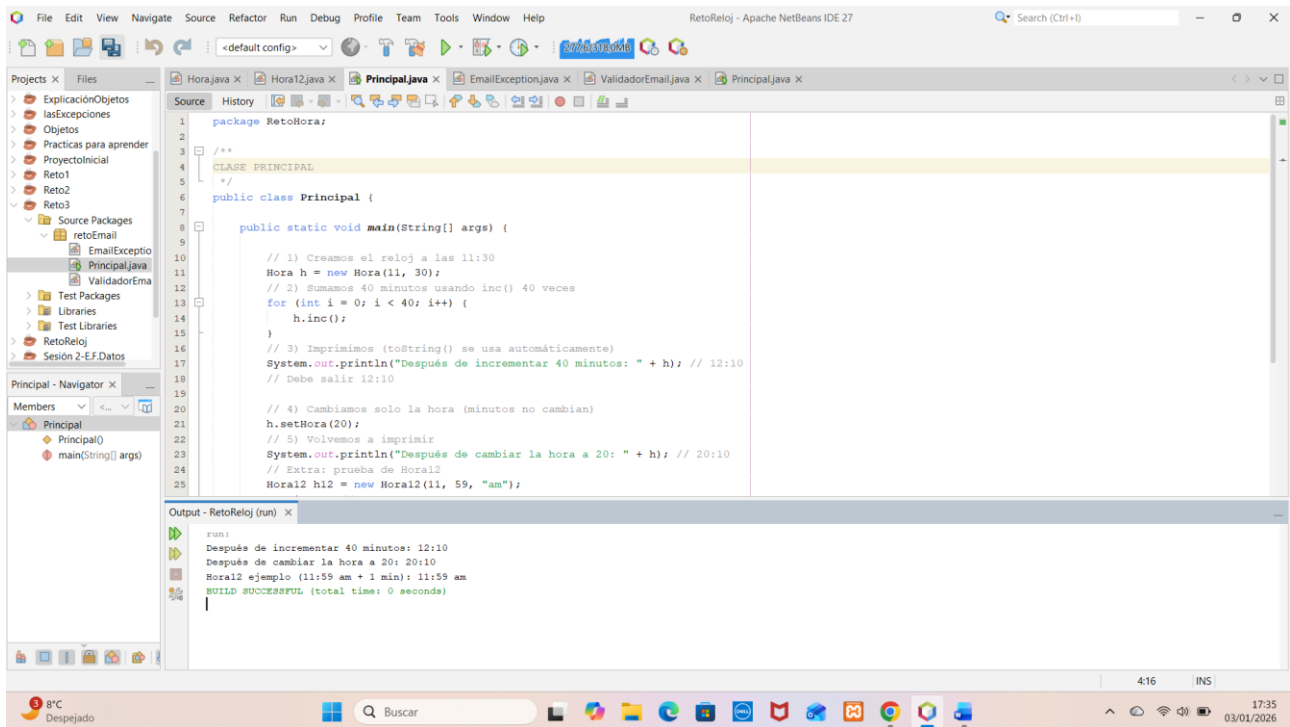
            // Si estamos en 11 y pasamos de hora, cambiamos am/pm
            if (getHora() == 11) {
                periodo = periodo.equals("am") ? "pm" : "am";
            }
            // Fórmula general para la hora en formato 12h
            // 1pasará2, 11pasará12, 12pasará1
            setHora((getHora() % 12) + 1);
        }
    }
}
```

```

    }
// Muestra: "hh:mm am/pm"
@Override
public String toString() {
    return String.format("%02d:%02d %s", getHora(), getMinuto(), periodo);
}
}

```

CLASE PRINCIPAL



CÓDIGO JAVA:

```

package RetoHora;

/**CLASE PRINCIPAL
 */
public class Principal {

    public static void main(String[] args) {

        // 1) Creamos el reloj a las 11:30
        Hora h = new Hora(11, 30);

        // 2) Sumamos 40 minutos usando inc() 40 veces
    }
}

```



```
for (int i = 0; i < 40; i++) {  
    h.inc();  
}  
// 3) Imprimimos (toString() se usa automáticamente)  
System.out.println("Después de incrementar 40 minutos: " + h); // 12:10  
// Debe salir 12:10  
  
// 4) Cambiamos solo la hora (minutos no cambian)  
h.setHora(20);  
// 5) Volvemos a imprimir  
System.out.println("Después de cambiar la hora a 20: " + h); // 20:10  
// Extra: prueba de Hora12  
Hora12 h12 = new Hora12(11, 59, "am");  
h12.inc(); // 12:00 pm  
System.out.println("Hora12 ejemplo (11:59 am + 1 min): " + h12);  
}  
}
```